Encryption at Rest and Key Management

encryption is easy; the keys are the hard part

Watch the 4-min video

In one line: encrypt stored data at the right layer, keep the master keys in a KMS or HSM that never releases them, and use envelope encryption: data keys encrypt data locally, the KMS only wraps and unwraps those small keys.

1 Is the stolen data readable?

A backup tape goes missing. A disk from a decommissioned server is sold online. A storage bucket is accidentally public. The question each time: can the finder read the data? Encryption at rest makes stolen storage useless, **as long as the key wasn't stored next to it**.

2 Layers of encryption at rest

Layer	Example	Protects against	Doesn't protect against
Full disk / volume	BitLocker, LUKS, EBS encryption	lost or stolen drives	anyone who can log in to the running machine
Database (TDE)	SQL Server / Azure SQL TDE, Oracle TDE	stolen data files and backups	a user or app with SQL access
Object storage	S3 SSE-KMS, Azure Storage encryption	media theft at the provider	leaked credentials, public buckets
Application / field level	encrypt card or ID numbers before INSERT	DB admins, SQL injection dumps, leaked backups	a compromised app that holds the key

Lower layers are transparent and cheap (S3 encrypts every new object by default since January 2023); higher layers protect against more attackers but cost search and indexing. Use several.

3 Keys live in a KMS

If the key sits next to the data, a thief gets both. Keys belong in a **key management service**: AWS KMS, Azure Key Vault, Google Cloud KMS. Master keys never leave it; applications ask it to perform operations, access is controlled by IAM, and every use is logged. But a KMS call is a network round trip with request quotas and a small payload limit (AWS KMS `Encrypt` takes at most 4 KB), so you don't send gigabytes through it.

4 Envelope encryption

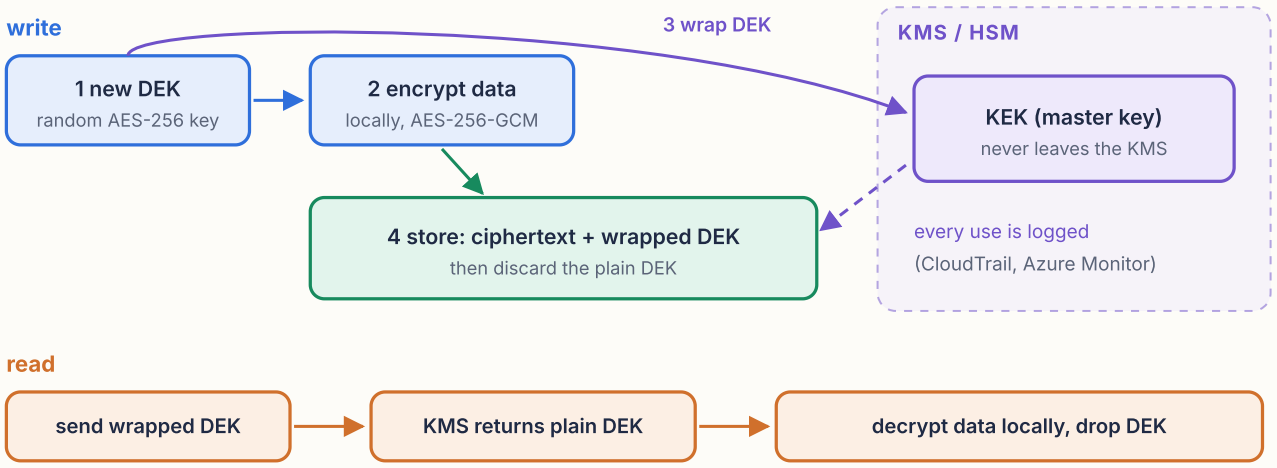


Figure 1. Envelope encryption: a data encryption key (DEK) encrypts the data locally; the key encryption key (KEK) in the KMS only wraps the DEK.

AWS's `GenerateDataKey` does steps 1 and 3 in one call, returning the plain DEK and the wrapped DEK. It is **fast** (bulk crypto is local, AES at GB/s), **scalable** (one DEK per object, file or tenant; cache plain DEKs briefly to cut KMS calls) and **cheap to rotate**: rotating the KEK means re-wrapping small DEKs, not re-encrypting terabytes. KMS-managed rotation keeps old key versions so old data still decrypts.

5 Customer-managed keys and HSMs

Customer-managed keys (CMK, sometimes BYOK) let a customer hold the master key in their own KMS account: revoke it, and the SaaS provider can no longer read their data. Many enterprises require this. **Hardware security modules** are tamper-resistant devices (validated to FIPS 140-2/140-3, often Level 3) for the highest-value keys: AWS CloudHSM, Azure Managed HSM. Rotate keys on a schedule (commonly yearly), and separate duties: the people who manage keys shouldn't also read the data.

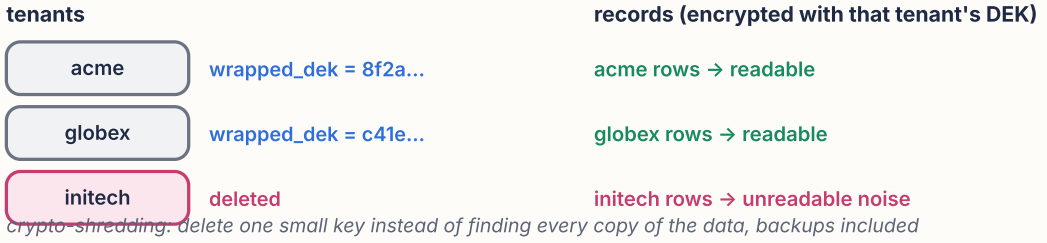


Figure 2. Per-tenant DEKs wrapped by one KMS master key. Deleting a tenant's wrapped key makes all their data unreadable at once.

WORKED EXAMPLE: PER-TENANT KEYS AND CRYPTO-SHREDDING

Each tenant row stores `wrapped_dek`, created with `GenerateDataKey` under the master key. Records are encrypted with AES-256-GCM using that tenant's DEK (with the tenant ID as associated data, so rows can't be swapped between tenants). When a tenant leaves, the service deletes the wrapped key: every record, replica and backup of theirs becomes unreadable. If one DEK ever leaks, only one tenant is exposed.

COMMON MISTAKES

- Keys in config files or in the same database as the data.
- Believing disk or TDE encryption protects against SQL injection or a stolen app credential; it doesn't.
- Deleting old key versions after rotation, so old backups can never be restored.
- Home-made crypto: ECB mode, reused GCM nonces, unauthenticated encryption.

6 Key terms

DEK: data encryption key; encrypts the actual data

KEK: key encryption key; wraps DEKs, stays in the KMS

KMS: managed service that stores keys and performs key operations

HSM: tamper-resistant hardware that holds keys

Customer-managed key: master key controlled by the customer

Crypto-shredding: destroying data by deleting its key

7 Interview questions

Q1. Explain envelope encryption. BASIC

Data is encrypted locally with a data encryption key; the DEK is wrapped by a key encryption key that never leaves the KMS, and only the wrapped DEK is stored with the data. To read, the KMS unwraps the DEK and decryption happens locally.

Q2. Why use envelope encryption? BASIC

Fast local encryption, per-object or per-tenant keys, and cheap master-key rotation: re-wrap the small DEKs, not the data.

Q3. The database volume is encrypted. Is customer data safe from a SQL injection dump? SCENARIO

No. Disk and TDE encryption are transparent to anyone querying the database. Use parameterised queries, and field-level encryption for the most sensitive columns.

Q4. An enterprise customer wants to be able to cut off your access to their data. How? SCENARIO

Customer-managed keys: their KEK lives in their KMS account and wraps their DEKs; if they revoke your access to it, you can't unwrap anything.

Q5. How do you rotate a master key without re-encrypting petabytes? TRICKY

With envelope encryption you only re-wrap DEKs under the new KEK version (or keep old versions for decryption and use the new one for new data).

Q6. How do you guarantee a deleted customer's data is gone from backups too? TRICKY

Crypto-shredding: encrypt each customer's data with their own key and delete the key; copies in backups become unreadable without editing the backups.

Next: SD-M-010 · Secrets Management backendroadmap.com